

Tag 5 - Eine neue Mathematik: endliche Körper

Gestern hast du mit CRC etwas Erstaunliches getan: du hast **mit Polynomen gerechnet, modulo 2**. Heute gehen wir einen Schritt weiter und entdecken, dass dahinter eine ganze mathematische Welt steckt: die der **endlichen Körper**. Das klingt sperrig, ist aber im Grunde nur „Modulo-Rechnung, hochgezogen auf Polynome“. Und genau diese Welt brauchen wir, um morgen Reed-Solomon zu verstehen – das Verfahren, das im Datamatrix-Code steckt.

Lernziele für heute

Am Ende von Tag 5 kannst du ...

- erklären, was ein **Körper** in der Mathematik ist und warum „Körper“ hier nicht „Volumen“ heißt
- die Rechenregeln in **GF(2)** (dem kleinsten Körper) anwenden
- zeigen, warum **Modulo 256** für Bytes nicht funktioniert
- in **GF(2³)** addieren und multiplizieren – mit Bleistift und Zettel
- für jedes Element in GF(2³) das **multiplikative Inverse** finden
- in Python eine kleine Klasse für GF(2³) bauen und damit experimentieren

Material: Bleistift, kariertes Papier, viel Geduld für die Multiplikationstafel.
Optional: zwei verschiedene Farben (für Datenbits und Polynom-Koeffizienten).

Block 1 · Was ist eigentlich ein „Körper“? (≈ 25 min)

Bisher haben wir mit ganz unterschiedlichen Zahlensystemen gerechnet: ganze Zahlen, modulo 10 (EAN-13), modulo 11 (ISBN), modulo 2 (Parität), Polynome modulo 2 (CRC). Lass uns mal sortieren, was die alle gemeinsam haben.

In jedem dieser Systeme kannst du:

- **Addieren** (immer)
- **Subtrahieren** (immer; bei Modulo wird's „addieren mit Gegenzahl“)
- **Multiplizieren** (immer)
- **Dividieren** – das ist die spannende Frage!

Wann kann man dividieren?

Dividieren heißt: zu jeder Zahl $\neq 0$ gibt es eine **Gegenzahl** (das *multiplikative Inverse*), sodass $a \cdot a^{-1} = 1$. Bei den reellen Zahlen ist das einfach: $5^{-1} = 0,2$. Bei modulo-Systemen wird's interessant.

Wann gibt es Inverse?



- (a) Versuche, in **modulo 10** für jede Zahl 1 bis 9 ein Inverses zu finden, also eine Zahl b , sodass $a \cdot b \equiv 1 \pmod{10}$. Welche Zahlen haben Inverse, welche nicht?

Tipp: Probiere systematisch. Für $a = 3$ z. B. die Reihe $3 \cdot 1 = 3$, $3 \cdot 2 = 6$, $3 \cdot 3 = 9$, $3 \cdot 4 = 12 \equiv 2$, ... und schaue, wann die 1 auftaucht.

- (b) Mache dasselbe für **modulo 11**. Welche Zahlen haben Inverse?
- (c) Was fällt dir auf? Welche Eigenschaft hat 11, die 10 nicht hat?
- (d) **Faustregel formulieren:** Bei Modulo n hat eine Zahl a genau dann ein Inverses, wenn ...?

Was ist nun ein Körper?

Ein **Körper** (engl. *field*) ist eine Menge von Zahlen, in der man **uneingeschränkt rechnen** kann: $+$, $-$, $\cdot$, $\div$ (außer Division durch 0). Konkret heißt das:

- Es gibt eine 0 (neutrales Element der Addition).
- Es gibt eine 1 (neutrales Element der Multiplikation).
- Jede Zahl hat eine Gegenzahl (additives Inverses).
- Jede Zahl $\neq 0$ hat ein multiplikatives Inverses.
- Plus die üblichen Rechenregeln (Assoziativität, Kommutativität, Distributivität).

Beispiele für Körper: - $\mathbb{R}$ (reelle Zahlen) - $\mathbb{Q}$ (rationale Zahlen) - $\mathbb{C}$ (komplexe Zahlen) - $\mathbb{Z}/p\mathbb{Z}$ – die ganzen Zahlen modulo p mit p prim (das ist neu!)

Beispiele für **keine** Körper: - $\mathbb{Z}$ (ganze Zahlen) – kein Inverses für 2 (es gibt kein „ $1/2$ “ in $\mathbb{Z}$) - $\mathbb{Z}/10\mathbb{Z}$ – modulo 10, weil 2, 4, 5, 6, 8 keine Inverse haben

Merke: **Körper modulo n gibt es nur, wenn n eine Primzahl ist.**
Diese Körper schreibt man oft auch $\text{GF}(p)$ – „Galois-Feld“ der Größe p , benannt nach dem französischen Mathematiker Évariste Galois (1811–1832, lebte nur 20 Jahre, hat aber eine ganze Disziplin der Mathematik begründet).

Der kleinste Körper: $\text{GF}(2)$

$\text{GF}(2)$ ist der aller kleinste Körper. Er hat genau zwei Elemente: 0 und 1. Du kennst ihn schon – nur unter einem anderen Namen.

- (a) Erstelle die **Additionstafel** für $\text{GF}(2)$:

	0	1
0		
1		

- (b) Erstelle die **Multiplikationstafel** für $\text{GF}(2)$:



Bleistift-
Übung 2

	0	1
0		
1		

- (c) Vergleiche: Welche aus Python bekannten Operationen ergeben dieselben Tabellen?

Tipp: Erinnere dich an Tag 1 (Parität) und Tag 4 (CRC).

- (d) Was ist $1 - 1$ in $\text{GF}(2)$? Was ist also die Gegenzahl von 1? (Gemeinheit: in $\text{GF}(2)$ ist das nicht „ -1 “.)

Block 2 · Warum Modulo 256 nicht der Weg ist (≈ 15 min)

Wir wollen mit Bytes rechnen, also mit Zahlen 0 bis 255. Naheliegender wäre: einfach Modulo 256. Aber 256 ist keine Primzahl ($256 = 2^8$), also haben wir das Problem aus Bleistiftübung 1 zurück.



Modulo 256 ist kaputt

Bleistift-
Übung 3

- (a) Was ist $2 \cdot 128 \bmod 256$? Folgerung: hat 2 ein Inverses in modulo 256?
- (b) Welche Zahlen aus 1..255 haben kein Inverses modulo 256?

Tipp: Eine Zahl a hat genau dann ein Inverses modulo n , wenn a und n keinen gemeinsamen Teiler außer 1 haben (man sagt: *teilerfremd* oder *koprim*).

- (c) Wie viele Zahlen aus 1..255 sind koprim zu 256? (Du musst nicht alle aufzählen, aber überlege, welche es nicht sind.)

Zwei mögliche Auswege

Wir hätten zwei Optionen:

Option 1: Statt 256 eine Primzahl in der Nähe nehmen, z. B. 257 (das ist tatsächlich prim!). Damit hätten wir $\text{GF}(257)$ – einen schönen Körper. Aber: Bytes sind 256 Werte, nicht 257. Wir würden ständig den 257-ten Wert „übrig“ haben. Unschön.

Option 2: Eine ganz neue Konstruktion: statt mit Zahlen modulo 2^8 rechnen wir mit **Polynomen** vom Grad < 8 über $\text{GF}(2)$, und nehmen modulo eines bestimmten Polynoms. Klingt fies, ist aber elegant – und genau das, was wir heute aufbauen.

Wir starten klein: erst mit $\text{GF}(2^3)$ (8 Elemente), dann später mit $\text{GF}(2^8)$ (256 Elemente).

Block 3 · GF(2³) konkret aufbauen (≈ 60 min)

Das ist der schwierigste Teil heute. Aber wenn du ihn verstanden hast, hast du das Tor zu Reed-Solomon weit aufgestoßen.

Die Grundidee

Wir betrachten **Polynome vom Grad kleiner als 3, deren Koeffizienten 0 oder 1 sind**. Das sind genau $2^3 = 8$ Stück:

0
1
x
x+1
x²
x²+1
x²+x
x²+x+1

Diese 8 Polynome sind die Elemente von GF(2³). Jedes lässt sich als 3-Bit-Zahl schreiben:

0	=	000
1	=	001
x	=	010
x+1	=	011
x ²	=	100
x ² +1	=	101
x ² +x	=	110
x ² +x+1	=	111

Praktischer Trick: Wir können also einfach mit 3-Bit-Zahlen rechnen, müssen aber die Rechenregeln der Polynome beachten.

Addition in GF(2³)

Polynome addieren wir koeffizientenweise. Da die Koeffizienten in GF(2) liegen (also 0 oder 1), ist die Addition modulo 2 – genau **XOR**.

Beispiel: $(x^2+x+1) + (x^2+1)$. Koeffizienten:

	1	1	1	
⊕	1	0	1	
	0	1	0	→ x

Also: $(x^2+x+1) + (x^2+1) = x$. Schreibst du das als 3-Bit-Zahlen: $111 \oplus 101 = 010$.



Bleistift-
Übung 4

Addition in $\text{GF}(2^3)$

Berechne (am einfachsten als XOR der 3-Bit-Darstellungen):

Aufgabe	Ergebnis (3-Bit)	Als Polynom
$(x+1) + (x^2+x)$		
$(x^2+1) + (x^2+x+1)$		
$x^2 + x^2$		
$(x^2+x+1) + (x^2+x+1)$		

Beobachtung: Was ist $a + a$ für jedes Element a ?

Multiplikation - jetzt wird's interessant

Polynome multiplizieren wir wie in der Schule, aber **die Koeffizienten in $\text{GF}(2)$** . Beispiel:

$$\begin{aligned}
 &(x + 1) \cdot (x + 1) \\
 &= x \cdot x + x \cdot 1 + 1 \cdot x + 1 \cdot 1 \\
 &= x^2 + x + x + 1 \\
 &= x^2 + 0 + 1 \quad (\text{denn } x + x = 0 \text{ in } \text{GF}(2)) \\
 &= x^2 + 1
 \end{aligned}$$

Aber: was, wenn das Ergebnis Grad ≥ 3 hat? Dann passt es nicht mehr in unsere 3-Bit-Welt.

$$(x^2) \cdot (x) = x^3 \quad \leftarrow \text{Grad 3, zu hoch!}$$

Modulo-Polynom: das irreduzible Polynom

Hier kommt der Schlüssel-Trick. So wie bei modulo p alles, was $\geq p$ wird, modulo p reduziert wird, nehmen wir hier **alles modulo eines bestimmten Polynoms** $p(x)$ mit Grad 3.

Welches Polynom nehmen wir? Es muss **irreduzibel** sein – das heißt, es darf nicht in Faktoren niedrigeren Grades zerlegbar sein (analog zu „Primzahl“ bei Zahlen).

Ein irreduzibles Polynom vom Grad 3 über $\text{GF}(2)$ ist:

$$p(x) = x^3 + x + 1$$

(Es gibt noch genau ein anderes, $x^3 + x^2 + 1$, aber das nehmen wir nicht.)

Modulo $p(x)$ rechnen

Wenn ein Multiplikationsergebnis ein x^3 enthält, ersetzen wir es: aus $p(x) = x^3 + x + 1 = 0$ (in unserer neuen Welt!) folgt $x^3 = x + 1$. Diese **Reduktionsregel** brauchen wir ständig:

$$x^3 = x + 1$$

$$x^4 = x \cdot x^3 = x \cdot (x + 1) = x^2 + x$$

$$x^5 = x \cdot x^4 = x \cdot (x^2 + x) = x^3 + x^2 = (x+1) + x^2 = x^2 + x + 1$$

Multiplikation in $GF(2^3)$



Bleistift-
Übung 5

Berechne, mit $p(x) = x^3 + x + 1$:

(a) $x \cdot x^2$ (kleine Aufwärmübung)

(b) $x^2 \cdot x^2$

Tipp: $x^2 \cdot x^2 = x^4$, und $x^4 = x^2 + x$ (siehe oben).

(c) $(x+1) \cdot (x^2+x)$

(d) $(x^2+x+1) \cdot (x^2+1)$

Tipp: erst auspolieren (Distributivgesetz), dann reduzieren.

Die vollständige Multiplikationstafel

Wenn du Lust hast, kannst du dir die komplette 8×8 -Multiplikationstafel von $GF(2^3)$ selbst ausrechnen. Das ist mühsam, aber sehr lehrreich – und du machst es nur einmal.

□ **Bleistiftübung 6 (Bonus) - Die Tafel** Fülle die Tafel aus. Element 0 lasse ich dir geschenkt, das ist die ganze erste Zeile/Spalte.

·	1	x	x+1	x ²	x ² +1	x ² +x	x ² +x+1
1	1	x	x+1	x ²	x ² +1	x ² +x	x ² +x+1
x	x						
x+1							
x ²							
x ² +1							
x ² +x							
x ² +x+1							

Wenn du das durchhältst, hast du mehr über endliche Körper gelernt als in den meisten Mathematik-Erstsemestern.

Inverse finden

Sobald du die Multiplikationstafel hast, sind die Inverse einfach abzulesen: das Inverse von a ist das Element b , sodass $a \cdot b = 1$.

Inverse aus der Tafel

Lies aus deiner Tafel ab:

Element	Inverses
1	
x	
x+1	
x ²	
x ² +1	
x ² +x	
x ² +x+1	

Wenn alles geklappt hat: **jedes Element $\neq 0$ hat ein Inverses**. Das ist genau das, was einen Körper ausmacht.



Bleistift-
Übung 7

Block 4 · Python-Werkstatt: GF(2³) implementieren (≈ 30 min)

Jetzt bauen wir das in Python. Lege eine neue Datei gf8.py an.

Die Klasse



Python-
Einheit 1

```
class GF8:
    """Element von GF(2^3), repräsentiert als 3-Bit-Zahl 0..7.

    Polynom-Bits: bit 0 = 1-Term, bit 1 = x-Term, bit 2 = x^2-Term.
    Modulo-Polynom: x^3 + x + 1, also als Zahl: 0b1011 = 11.
    """

    MOD = 0b1011  # x^3 + x + 1

    def __init__(self, wert):
        if not 0 <= wert <= 7:
            raise ValueError("GF8-Werte müssen zwischen 0 und 7 liegen")
        self.wert = wert

    def __add__(self, anderes):
        """Addition = XOR."""
        return GF8(self.wert ^ anderes.wert)

    def __sub__(self, anderes):
        """In GF(2^n) ist Subtraktion = Addition."""
        return self + anderes
```

```

def __mul__(self, anderes):
    """Polynom-Multiplikation modulo  $x^3 + x + 1$ ."""
    a, b = self.wert, anderes.wert
    ergebnis = 0
    # Schulmultiplikation: für jedes Bit von b
    for i in range(3):
        if (b >> i) & 1:
            ergebnis ^= a << i
    # Reduktion: solange Grad >= 3, mit MOD reduzieren
    for i in range(5, 2, -1): # höchstes Bit ist 5 (Bit 2 von a · Bit 2 von b)
        if (ergebnis >> i) & 1:
            ergebnis ^= self.MOD << (i - 3)
    return GF8(ergebnis)

def __eq__(self, anderes):
    return self.wert == anderes.wert

def __repr__(self):
    return f"GF8({self.wert:03b})"

# Tests
a = GF8(0b011) # x+1
b = GF8(0b110) # x^2+x
print(a + b) # erwartet: 0b101 = x^2+1
print(a * b) # erwartet: rechne von Hand und vergleiche!

```

Aufgabe 1.1 - Verifiziere mit deiner Bleistift-Tafel Wähle drei Multiplikationen aus Bleistiftübung 5 oder aus deiner Tafel und prüfe, ob das Python-Ergebnis übereinstimmt.

Inverse per Brute Force



Python-
Einheit 2

Solange wir nur 8 Elemente haben, können wir das Inverse jedes Elements einfach durch Probieren finden:

```

def gf8_inverse(a):
    """Findet das multiplikative Inverse von a in GF(2^3) durch Probieren."""
    if a.wert == 0:
        raise ValueError("0 hat kein Inverses")
    eins = GF8(1)
    for kandidat in range(1, 8):
        if a * GF8(kandidat) == eins:
            return GF8(kandidat)
    raise RuntimeError("Kein Inverses gefunden – sollte nicht passieren!")

# Teste alle Elemente
for w in range(1, 8):
    a = GF8(w)
    inv = gf8_inverse(a)
    print(f"{a}^(-1) = {inv}, Probe: {a} · {inv} = {a * inv}")

```

Aufgabe 2.1 - Vergleiche mit deiner Tafel Stimmen die berechneten Inverse mit Bleistiftübung 7 überein?

Aufgabe 2.2 - Multiplikationstafel automatisch erzeugen Schreibe Code, der die komplette 8×8 -Multiplikationstafel ausgibt. Format zum Beispiel:

```

· | 0  1  2  3  4  5  6  7
---+-----
0 | 0  0  0  0  0  0  0  0
1 | 0  1  2  3  4  5  6  7
...

```

Vergleiche das mit deiner Bleistift-Tafel aus Bleistiftübung 6 (falls du sie gemacht hast).

Block 5 · Reflexion und Ausblick (≈ 15 min)

Was wir heute gelernt haben

- **Körper** sind Zahlensysteme, in denen man uneingeschränkt rechnen kann (auch dividieren, außer durch 0).
- **$\text{GF}(p)$** für p prim ist ein Körper – einfach Modulo-Rechnung mit Inversen.
- **Modulo 256 ist kein Körper** – man kann nicht teilen.
- Die Lösung: **Polynom-Arithmetik modulo eines irreduziblen Polynoms** liefert Körper für jede Zahl der Form 2^n .
- **$\text{GF}(2^3)$** ist der erste solche Körper, den wir konkret aufgebaut haben: 8 Elemente, jedes ein Polynom vom Grad < 3 , Multiplikation modulo $x^3 + x + 1$.
- **Jedes Element $\neq 0$ hat ein Inverses** – wir haben die Inverse durch Probieren gefunden.

Drei Fragen zum Mitnehmen

1. Wir haben $\text{GF}(2^3)$ gemacht. Für Bytes brauchen wir $\text{GF}(2^8)$ – also 256 Elemente, Polynome vom Grad < 8 modulo eines irreduziblen Polynoms vom Grad 8. Wie würde sich der Code aus Python-Einheit 1 ändern? (Tipp: vier Stellen, alle minimal.)
2. Inverse durch Probieren ist ineffizient. Bei $\text{GF}(2^8)$ wären das 255 Versuche pro Inverse. Es gibt einen viel besseren Algorithmus (genannt **Erweiterter Euklidischer Algorithmus**). Bevor du davon erfährst – wie würdest *du* spontan vorschlagen, das schneller zu machen?
3. Wir haben heute viele Polynome auf einer abstrakten Ebene behandelt – als Elemente eines Körpers. Ab morgen werden wir Polynome aber wieder als „echte“ Polynome verwenden, deren Koeffizienten *Elemente* eines Körpers sind. Klingt verwirrend? Es ist genau wie bei reellen Zahlen: $\mathbb{R}$ ist ein Körper, und ein Polynom wie $3x^2 + 5x - 2$ hat Koeffizienten *aus* $\mathbb{R}$. Wir machen jetzt dasselbe, nur mit $\text{GF}(2^8)$ statt $\mathbb{R}$.

Vorschau Tag 6

Morgen treffen wir auf die zentrale Reed-Solomon-Idee: **Daten als Polynom interpretieren, an mehreren Stellen auswerten, und genug zusätzliche Auswertungen mitspeichern, um aus weniger als der Hälfte aller Stellen das Original rekonstruieren zu können.** Das ist die geniale Verallgemeinerung dessen, was wir bei ISBN-10 mit „eine Ziffer rekonstruieren“ schon gemacht haben.

Dafür brauchen wir die Werkzeuge von heute (Körper-Arithmetik) und die Werkzeuge von Tag 4 (Polynomdivision). Tag 6 ist also Synthese der letzten zwei Tage.

Lösungen (erst nach eigenem Versuch ansehen!)

Bleistiftübung 1 - Inverse modulo 10 und 11

(a) Modulo 10:

a	Inverse	Rechnung
1	1	$1 \cdot 1 = 1$
2	-	keine: $2 \cdot k$ ist immer gerade, kann nie $\equiv 1 \pmod{10}$ sein
3	7	$3 \cdot 7 = 21 \equiv 1$
4	-	gleich wie 2
5	-	$5 \cdot k$ endet auf 0 oder 5
6	-	gleich wie 2
7	3	$7 \cdot 3 = 21 \equiv 1$
8	-	gleich wie 2
9	9	$9 \cdot 9 = 81 \equiv 1$

Inverse haben: 1, 3, 7, 9 - also genau die zu 10 teilerfremden Zahlen.

(b) Modulo 11:

a	Inverse
1	1
2	6
3	4
4	3
5	9
6	2
7	8
8	7
9	5
10	10

Alle haben Inverse.

(c) 11 ist eine Primzahl, 10 nicht.

(d) **Faustregel:** Eine Zahl a hat genau dann ein Inverses modulo n , wenn a und n teilerfremd sind (d. h. $\text{ggT}(a,n) = 1$). Folgt daraus: ist n prim, dann sind alle Zahlen $1..n-1$ zu n teilerfremd $\rightarrow$ alle haben Inverse.

Bleistiftübung 2 - GF(2)

- (a) Additionstafel: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=0$.
- (b) Multiplikationstafel: $0\cdot 0=0$, $0\cdot 1=0$, $1\cdot 0=0$, $1\cdot 1=1$.
- (c) Addition entspricht **XOR**, Multiplikation entspricht **AND**.
- (d) $1 - 1 = 1 + 1 = 0$ (denn die Subtraktion ist die Addition mit der Gegenzahl, und in GF(2) ist die Gegenzahl von 1 die 1 selbst). Allgemeines Prinzip: in GF(2^n) ist jede Zahl ihre eigene Gegenzahl.

Bleistiftübung 3 - Modulo 256

- (a) $2 \cdot 128 \bmod 256 = 256 \bmod 256 = 0$. Also: 2 hat kein Inverses (denn wenn wir mit 2 multiplizieren und etwas erhalten, das durch 256 teilbar ist, nämlich 0, kann das Ergebnis nicht 1 sein).
- (b) Alle **geraden** Zahlen aus 1..255 haben kein Inverses, weil sie mit 256 den Faktor 2 gemeinsam haben.
- (c) Es gibt 128 ungerade Zahlen (1, 3, 5, ..., 255) – nur diese sind koprim zu 256.

Bleistiftübung 4 - Addition in GF(2^3)

Aufgabe	3-Bit (XOR)	Polynom
$(x+1) + (x^2+x)$	$011 \oplus 110 = 101$	x^2+1
$(x^2+1) + (x^2+x+1)$	$101 \oplus 111 = 010$	x
$x^2 + x^2$	$100 \oplus 100 = 000$	0
$(x^2+x+1) + (x^2+x+1)$	$111 \oplus 111 = 000$	0

Beobachtung: $a + a = 0$ für jedes Element. Jedes Element ist seine eigene Gegenzahl.

Bleistiftübung 5 - Multiplikation in GF(2^3)

- (a) $x \cdot x^2 = x^3$. Reduziere mit $x^3 = x + 1$: Ergebnis = **$x + 1$** (= 011).
- (b) $x^2 \cdot x^2 = x^4 = x^2+x$ (= 110).
- (c) $(x+1) \cdot (x^2+x)$:

$$\begin{aligned}
 &= x \cdot (x^2+x) + 1 \cdot (x^2+x) \\
 &= x^3+x^2 + x^2+x \\
 &= x^3 + (x^2 + x^2) + x \\
 &= x^3 + 0 + x \\
 &= x^3 + x
 \end{aligned}$$

Reduziere $x^3 \rightarrow x+1$:

$$= (x+1) + x = 2x + 1 = 0 \cdot x + 1 = 1$$

Ergebnis: **1**. (Das heißt zugleich: $x+1$ und x^2+x sind zueinander invers.)

(d) $(x^2+x+1) \cdot (x^2+1)$:

$$= x^2 \cdot (x^2+1) + x \cdot (x^2+1) + 1 \cdot (x^2+1)$$

$$= x^4+x^2 + x^3+x + x^2+1$$

$$= x^4 + x^3 + (x^2+x^2) + x + 1$$

$$= x^4 + x^3 + 0 + x + 1$$

Reduziere $x^4 \rightarrow x^2+x$ und $x^3 \rightarrow x+1$:

$$= (x^2+x) + (x+1) + x + 1$$

$$= x^2 + (x+x+x) + (1+1)$$

$$= x^2 + x + 0$$

$$= x^2 + x$$

Ergebnis: **$x^2 + x$** (= 110).

Bleistiftübung 6 - Multiplikationstafel $\text{GF}(2^3)$

·	1	x	x+1	x ²	x ² +1	x ² +x	x ² +x+1
1	1	x	x+1	x ²	x ² +1	x ² +x	x ² +x+1
x	x	x ²	x ² +x	x+1	1	x ² +x+1	x ² +1
x+1	x+1	x ² +x	x ² +1	x ² +x+1	x ²	1	x
x ²	x ²	x+1	x ² +x+1	x ² +x	x	x ² +1	1
x ² +1	x ² +1	1	x ²	x	x ² +x+1	x+1	x ² +x
x ² +x	x ² +x	x ² +x+1	1	x ² +1	x+1	x	x ²
x ² +x+1	x ² +x+1	x ² +1	x	1	x ² +x	x ²	x+1

(Einzelne Felder zur Selbstkontrolle: $x \cdot x^2 = x^3 \rightarrow x+1$ ✓, $(x+1)(x^2+x) = 1$ ✓, $(x^2+1) \cdot x = x^3+x = (x+1)+x = 1$ ✓.)

Element	Inverses
---------	----------

Bleistiftübung 7 - Inverse aus der Tafel

Element	Inverses
1	1
x	x^2+1
$x+1$	x^2+x
x^2	x^2+x+1
x^2+1	x
x^2+x	$x+1$
x^2+x+1	x^2

Probe: $x \cdot (x^2+1) = x^3 + x = (x+1) + x = 1 \checkmark$.

Aufgabe 1.1, 2.1, 2.2 (Python)

Erwartete Übereinstimmung mit der Hand-Tafel. Der häufigste Fehlerquell ist die Reduktionsschleife in `__mul__` – wenn die Bit-Indizes nicht stimmen, kommt Müll raus. Beim Debuggen lohnt es sich, ein konkretes Beispiel von Hand und in Python parallel laufen zu lassen.

Antworten zu den drei Reflexionsfragen

1. GF(2⁸) statt GF(2³): vier Änderungen:

- MOD wird zu einem Polynom vom Grad 8, z. B. 0b100011101 (das ist $x^8+x^4+x^3+x^2+1$, das standardmäßig für AES und Reed-Solomon im Datamatrix verwendete irreduzible Polynom).
- Wertebereich-Check 0..7 wird zu 0..255.
- Die Schleife `for i in range(3)` wird zu `for i in range(8)`.
- Die Reduktionsschleife `for i in range(5, 2, -1)` wird zu `for i in range(14, 7, -1)` (denn das Produkt zweier Grad-7-Polynome hat höchstens Grad 14).

2. Schnellere Inverse:

Eine offensichtliche Idee ist eine **Tabelle** vorzuberechnen (geht bei nur 256 Elementen sehr gut – brauche nur 256 Bytes Speicher). Eine andere: den **Satz von Fermat** nutzen, der besagt $a^{(2^n-1)} = 1$ in $GF(2^n)$, also $a^{(2^n-2)} = a^{-1}$. Für $GF(2^8)$: $a^{254} = a^{-1}$. Das berechnet man mit „schneller Potenzierung“ in nur 8 Multiplikationen statt 255 Versuchen. Der Erweiterte Euklidische Algorithmus ist die mathematisch eleganteste Lösung.

3. Diese Frage ist eine reine Vorschau. Die Idee „Polynome mit Koeffizienten aus einem Körper“ ist tatsächlich der Schlüssel zu Tag 6: ein RS-Codewort ist ein Polynom mit Koeffizienten in $GF(2^8)$.